



Universidad Nacional Autónoma de México
Facultad de Ingeniería
Inteligencia Artificial

Tarea #1 : DFS y BFS aplicados al Metro de la Ciudad de México

Alumno: Vera Morales Juan Pablo

Grupo: 10

Profesor : Romero de la Cruz Alejandro

Introducción

El siguiente reporte comprende la implementación y análisis de los algoritmos de búsqueda BFS (Breadth-First Search) y DFS (Depth-First Search) con el propósito de reforzar los conceptos vistos en clase y analizar la completitud, optimalidad y complejidad tanto en tiempo como en espacio de ambos algoritmos basados en distintas métricas aplicándolos a un grafo construido a partir del mapa del Metro CDMX.

Herramientas

Para la realización de la tarea se utilizó el lenguaje Python, un archivo JSON para la carga de datos y el editor de código Visual Studio Code.

Desarrollo

1. Construcción del grafo del Metro CDMX

El Metro de la Ciudad de México se modeló como un grafo no dirigido $G = (V, E)$, donde el conjunto de nodos V es representado por las estaciones y el conjunto de aristas representa las conexiones directas entre estaciones dentro de cada línea. El grafo fue implementado mediante una lista de adyacencia utilizando un diccionario, donde cada estación es una clave y su valor es la lista de estaciones vecinas.

Ejemplo:

```
30 print(metro["Tacubaya"])
31
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Z/XOS CONSOLE

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 1\grafoMetro.py"
['Observatorio', 'Juanacatlan', 'Constituyentes', 'San Pedro de los Pinos', 'Patriotismo']
```



Del diccionario “metro” con la key “Tacubaya” tenemos en la lista de adyacencia: ['Observatorio', 'Juanacatlán', 'Constituyentes', 'San Pedro de los Pinos', 'Patriotismo']

1.1 ¿Por qué utilizar lista de adyacencia en lugar de matriz de adyacencia?

El cálculo del costo en memoria de la lista de adyacencia es:

$$O(V + E) = 195(\text{número de estaciones}) + 2-4(\text{aproximado de conexiones por estación})$$

$$O(V + E) = 197$$

Mientras que en la matriz:

$$O(V^2) = 195 * 195 = 38,025$$

Esto pasa porque la matriz de adyacencia guarda todas las posiciones aunque no haya conexión, es decir, no importa lo alejada que esté una estación de otra esta deberá tener una celda que guarde la no-conexión de estas, por esto se prefirió la lista de adyacencia.

Carga de datos y estructura del .JSON

```
{
  "mapaMetro": {
    "1": [
      "Observatorio",
      "Tacubaya",
      "Juanacatlan",
      "Chapultepec",
      "Sevilla",
      "Insurgentes",
      "Cuauhtemoc",
      "Balderas",
      "Salto del Agua",
      "Isabel la Catolica",
      "Pino Suarez",
      "Merced",
      "Candelaria",
      "San Lazaro",
      "Moctezuma",
      "Balbuena",
      "Boulevard Puerto Aereo",
      "Gomez Farias",
      "Zaragoza",
      "Pantitlan"
    ],
  },
}
```

El archivo .JSON tiene la información de cada línea desplegada de la forma anterior, esta se carga de la siguiente forma.

```
import json

with open("mapaMetro.json","r",encoding="utf-8") as file:
    data = json.load(file)

mapaMetro = data["mapaMetro"]
```

Importamos el módulo .JSON , abrimos el archivo en modo de lectura y lo llamamos file, luego creamos “data” con la que convertimos el archivo de tipo .JSON a estructura de python, finalmente guardamos las líneas del metro en mapa Metro.

En este momento tenemos mapa Metro un diccionario que tiene como key el número o letra de la línea y como contenido las estaciones de la línea correspondiente:

```
print(mapaMetro)

[Running] python -u "c:\Users\lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 1\grafoMetro.py"
{'1': ['Observatorio', 'Tacubaya', 'Juanacatlan', 'Chapultepec', 'Sevilla', 'Insurgentes', 'Cuauhtemoc', 'Balderas', 'Salto del Agua', 'Isabel la Catolica', 'Pino Suarez', 'Merced', 'Candelaria', 'San Lazaro', 'Moctezuma', 'Balbuena', 'Boulevard Puerto Aereo', 'Gomez Farias', 'Zaragoza', 'Pantitlan'], '2': ['Cuatro Caminos', 'Panteones', 'Tacuba', 'Cuixtlahuac', 'Popotla', 'Colegio Militar', 'Normal', 'San Cosme', 'Revolucion', 'Hidalgo', 'Bellas Artes', 'Allende', 'Zocalo', 'Pino Suarez', 'San Antonio Abad', 'Chabacano', 'Viaducto', 'Xola', 'Villa de Cortes', 'Nativitas', 'Portales', 'Frelta', 'General Anaya', 'Tasqueña'], '3': ['Indios Verdes', 'Deportivo 18 de Marzo', 'Potrero', 'La Raza', 'Tlatelolco', 'Guerrero', 'Hidalgo', 'Juarez', 'Balderas', 'Ninos Heroes', 'Hospital General', 'Centro Medico', 'Etiopia', 'Eugenia', 'Division del Norte', 'Zapata', 'Coyoacan', 'Viveros', 'Miguel Angel de Quevedo', 'Copilco', 'Universidad'], '4': ['Martin Carrera', 'Talisman', 'Bondojito', 'Consulado', 'Canal del Norte', 'Morelos', 'Candelaria', 'Fray Servando', 'Jamaica', 'Santa Anita'], '5': ['Politecnico', 'Instituto del Petroleo', 'Autobuses del Norte', 'La Raza', 'Misterios', 'Valle Gomez', 'Consulado', 'Eduardo Molina', 'Aragon', 'Oceania', 'Terminal Aerea', 'Hangares', 'Pantitlan'], '6': ['El Rosario', 'Tezozomoc', 'Azcapotzalco', 'Ferretera', 'Norte 45', 'Vallejo', 'Instituto del Petroleo', 'Lindavista', 'Deportivo 18 de Marzo', 'La Villa-Baslica', 'Martin Carrera'], '7': ['El Rosario', 'Aguiles Serdan', 'Camarones', 'Refineria', 'Tacuba', 'San Joaquin', 'Polanco', 'Auditorio', 'Constituyentes', 'Tacubaya', 'San Pedro de los Pinos', 'San Antonio', 'Mixcoac', 'Barranca del Muerto'], '8': ['Garibaldi', 'Bellas Artes', 'San Juan de Letran', 'Salto del Agua', 'Doctores', 'Obrera', 'Chabacano', 'La Viga', 'Santa Anita', 'Coyuya', 'Iztacalco', 'Apatlaco', 'Aculco', 'Escuadron 201', 'Atlatilco', 'Iztapalapa', 'Cerro de la Estrella', 'UAM-I', 'Constitucion de 1917'], '9': ['Tacubaya', 'Patriotismo', 'Chilpancingo', 'Centro Medico', 'Lazaro Cardenas', 'Chabacano', 'Jamaica', 'Mixiuhca', 'Velodromo', 'Ciudad Deportiva', 'Puebla', 'Pantitlan'], 'A': ['Pantitlan', 'Agricola Oriental', 'Canal de San Juan', 'Tepalcates', 'Guelatao', 'Penon Viejo', 'Acetilla', 'Santa Marta', 'Los Reyes', 'La Paz'], 'B': ['Buena Vista', 'Guerrero', 'Garibaldi', 'Lagunilla', 'Tepito', 'Morelos', 'San Lazaro', 'Flores Magon', 'Rosero Rubio', 'Oceania', 'Deportivo Oceania', 'Bosque de Aragon', 'Villa de Aragon', 'Mazahualcoyotl', 'Impulsora', 'Rio de los Remedios', 'Muzquiz', 'Ecatepec', 'Olimpico', 'Plaza Aragon', 'Ciudad Azteca'], '12': ['Mixcoac', 'Insurgentes Sur', 'Hospital 20 de Noviembre', 'Zapata', 'Parque de los Venados', 'Eje Central', 'Ermita', 'Mexicaltzingo', 'Atlatilco', 'Cuahuacan', 'San Andres Tomatlan', 'Lomas Estrella', 'Calle 11', 'Periferico Oriente', 'Tezonco', 'Olivos', 'Nopalera', 'Zapotitlan', 'Tlaltemco', 'Tlahuac']}
```

Para pasar de un diccionario de líneas a un diccionario de estaciones tenemos lo siguiente:

```
12 metro = {}
13 |
14 for linea in mapaMetro.values():
15     for i in range(len(linea)):
16         estacion = linea[i]
17
18         if estacion not in metro:
19             metro[estacion] = []
20
21
22         if i > 0:
23             vecino = linea[i-1]
24             if vecino not in metro[estacion]:
25                 metro[estacion].append(vecino)
26
27
28         if i < len(linea) - 1:
29             vecino = linea[i+1]
30             if vecino not in metro[estacion]:
31                 metro[estacion].append(vecino)
```

Primero creamos un nuevo diccionario “metro”, este es el que tendrá como key el nombre de la estación y como contenido la lista de adyacencia de la estación, se rellena de la siguiente forma, creamos un ciclo for externo que itera por cada línea que hay dentro del diccionario “mapa Metro” , declaramos un ciclo for interno con un índice que opera según la longitud (cantidad de estaciones) de la linea, la estación actual es la estación dentro de la línea en el

índice “i” , si la estación todavía no es una key dentro del diccionario metro la creamos y le asignamos como valor una lista vacía.

Los siguientes condicionales añade las estaciones vecinas, si el índice no es 0 creamos la variable “vecino” que sera la estación en el índice anterior, si el vecino todavia no esta dentro de su lista de adyacencia lo añadimos, de misma forma si el índice es igual a la longitud de la línea - 1 , es decir no es la última estación de la línea creamos a vecino como la estación del índice siguiente, si todavia no esta en la lista de adyacencia lo añadimos.

Este ciclo se repetirá hasta haber recorrido todas las líneas de mapa Metro, creando así nuestro grafo de todas las estaciones con sus respectivas listas de adyacencia.

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 1\grafoMetro.py"
{'Observatorio': ['Tacubaya'], 'Tacubaya': ['Observatorio', 'Juanacatlan', 'Constituyentes', 'San Pedro de los Pinos', 'Patriotismo'], 'Juanacatlan': ['Tacubaya', 'Chapultepec'], 'Chapultepec': ['Juanacatlan', 'Sevilla'], 'Sevilla': ['Chapultepec', 'Insurgentes'], 'Insurgentes': ['Sevilla', 'Cuauhtemoc'], 'Cuauhtemoc': ['Insurgentes', 'Balderas'], 'Balderas': ['Cuauhtemoc', 'Salto del Agua', 'Juarez', 'Ninos Heroes'], 'Salto del Agua': ['Balderas', 'Isabel la Catolica', 'San Juan de Letran', 'Doctores'], 'Isabel la Catolica': ['Salto del Agua', 'Pino Suarez'], 'Pino Suarez': ['Isabel la Catolica', 'Merced', 'Zocalo', 'San Antonio Abad'], 'Merced': ['Pino Suarez', 'Candelaria'], 'Candelaria': ['Merced', 'San Lazaro', 'Morelos', 'Fray Servando'], 'San Lazaro': ['Candelaria', 'Moctezuma', 'Morelos', 'Flores Magon'], 'Moctezuma': ['San Lazaro', 'Balbuena'], 'Balbuena': ['Moctezuma', 'Boulevard Puerto Aereo'], 'Boulevard Puerto Aereo': ['Balbuena', 'Gomez Farias'], 'Gomez
```

2. Implementación de BFS y DFS

2.1 BFS

La implementación del algoritmo Breadth-First Search tiene la siguiente estructura:

```
1  from collections import deque
2
3  def bfs(grafo, inicio, fin):
4      frontera = deque([inicio])
5      explorados = set()
6      padre = {inicio: None}
```

Importamos del módulo collections la estructura “de que”, esta nos permite replicar la estructura FIFO donde lo que se encuentra al final de la cola (lo primero que entró) será lo primero en salir, comportamiento necesario para el algoritmo BFS, se define que la función tomará 3 parámetros, “grafo” nuestro diccionario con las estaciones y sus listas de adyacencia e “inicio” y “fin” respectivamente la estación desde la que comenzamos y a la que queremos llegar, frontera es nuestra cola FIFO quien contendrá los nodos (estaciones) a explorar y empieza con “inicio” dentro de ella, definimos explorados como un set lo que significa que no podrá contener valores repetidos evitandonos las repeticiones, finalmente padre es un diccionario que funcionara como nuestro grafo de recreación del la ruta de regreso, el nombre de la estacion será su key dentro del diccionario y lo que guarda es su nodo “padre”.

```

8      while frontera:
9          nodo = frontera.popleft()
10
11         if nodo == fin:
12             camino = []
13             while nodo is not None:
14                 camino.append(nodo)
15                 nodo = padre[nodo]
16             return camino[::-1], len(explorados)
17
18         explorados.add(nodo)
19
20
21         for vecino in grafo[nodo]:
22             if vecino not in explorados and vecino not in frontera:
23                 padre[vecino] = nodo
24                 frontera.append(vecino)
25
26     return None

```

El bucle principal “while frontera” se ejecuta mientras sigan quedando nodos por explorar, el nodo actual es aquel al que hacemos pop de la cola (el primero que entro), verificamos si ya estamos en el destino, si es así creamos la lista vacía “camino”, añadimos el nodo a la lista de “camino”, tomamos del diccionario de “padre” quien era su nodo antecesor y se convierte en el nuevo valor del nodo, esto se ejecuta hasta que llegamos a “inicio” el único nodo sin padre, una vez recreada esta ruta retornamos la lista “camino” desde el inicio hasta el final y dada la vuelta para que se imprima desde el nodo que iniciamos hasta en el que terminamos, retornamos también la longitud de “explorados” que es la cantidad de nodos que expandimos, en el caso de aun no encontrarnos en el final se itera por cada uno de los vecinos del nodo encontrado en la key “nodo” del grafo, si el vecino actual no está en explorados ni en la cola creamos la key del vecino dentro de “padre”, le asignamos el valor “nodo” dentro del diccionario y lo añadimos a la cola de frontera, finalmente si no se encontró ruta de una estación a otra retornamos None.

2.2 DFS

```

def dfs(grafo, inicio, fin):
    frontera = [inicio]
    explorados = set()
    padre = {inicio: None}

```

Depth-First Search cuenta con una estructura similar a BFS, la única diferencia es que este no requiere de una estructura FIFO, por lo que con una lista común bastará para simular la pila LIFO, “explorados” y “padre” se definen de la misma forma para recrear la cantidad de nodos expandidos y los nodos antecesores respectivamente.

```

8      while frontera:
9          nodo = frontera.pop()
10
11         if nodo == fin:
12
13             camino = []
14             while nodo is not None:
15                 camino.append(nodo)
16                 nodo = padre[nodo]
17             return camino[::-1], len(explorados)
18
19         if nodo not in explorados:
20             explorados.add(nodo)
21
22         for vecino in grafo[nodo]:
23             if vecino not in explorados:
24                 padre[vecino] = nodo
25                 frontera.append(vecino)
26
27     return None
28

```

El ciclo “while frontera” cumple la misma función, mientras siga habiendo contenido en la pila continuar iterando, hacemos pop de lo que esté hasta arriba de la pila dentro de la variable nodo, hacemos la verificación para saber si estamos en el nodo destino y con el mismo ciclo while reconstruimos mediante el diccionario padre el camino que se tomó para llegar hasta allí.

La primera gran diferencia con el código de BFS entra en la siguiente verificación:

```

if nodo not in explorados:
    explorados.add(nodo)

```

Como la estructura es una pila no sabe si el nodo actual se encuentra debajo del elemento que estoy revisando actualmente por lo que verificó en el set “explorados” si ya ha sido añadido, realizada esta verificación por cada vecino del nodo si el vecino aún no ha sido explorado lo añadimos al diccionario “padre” junto con el nodo actual como contenido indicando que es su padre y a hacemos push a la cola ese nodo vecino, el ultimo nodo vecino añadido será el próximo al que se hará pop de la cola “frontera”.

3. Pruebas

Para las pruebas se implementaron las siguientes dos funciones que presentan los resultados de ambos algoritmos en consola y de manera gráfica respectivamente:

```

import bfs
import dfs
import grafoMetro
import networkx as nx
import matplotlib.pyplot as plt

def pruebas(inicio, fin):

    # BFS
    rutaBFS, visitados_bfs = bfs.bfs(grafoMetro.metro, inicio, fin)
    print(f"\nResultados BFS desde {inicio} hasta {fin}:")
    if rutaBFS:
        print(" -> ".join(rutaBFS))
        print(f"Saltos: {len(rutaBFS) - 1}")
        print(f"Nodos expandidos: {visitados_bfs}")
    else:
        print("No se encontró ruta")

    # DFS
    rutaDFS, visitados_dfs = dfs.dfs(grafoMetro.metro, inicio, fin)
    print(f"\nResultados DFS desde {inicio} hasta {fin}:")
    if rutaDFS:
        print(" -> ".join(rutaDFS))
        print(f"Saltos: {len(rutaDFS) - 1}")
        print(f"Nodos expandidos: {visitados_dfs}")
    else:
        print("No hay una ruta")

    graficarRutas(grafoMetro.metro, rutaBFS, rutaDFS, inicio, fin)

```

```

def graficarRutas(mapaMetro, rutaBFS, rutaDFS, inicio, fin):

    G = nx.Graph()
    for estacion, vecinos in mapaMetro.items():
        for v in vecinos:
            G.add_edge(estacion, v)

    pos = nx.kamada_kawai_layout(G)

    fig, axes = plt.subplots(1, 2, figsize=(20, 10))

    # BFS
    ax = axes[0]
    ax.set_title(f"BFS: {inicio} -> {fin}")

    nx.draw_networkx_edges(G, pos, ax=ax, width=0.3, alpha=0.3)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=15)
    nx.draw_networkx_labels(G, pos, ax=ax, font_size=5)

    if rutaBFS:
        edges_bfs = [(rutaBFS[i], rutaBFS[i+1]) for i in range(len(rutaBFS)-1)]
        nx.draw_networkx_edges(G, pos, ax=ax, edgelist=edges_bfs, width=2)

    # DFS
    ax = axes[1]
    ax.set_title(f"DFS: {inicio} -> {fin}")

    nx.draw_networkx_edges(G, pos, ax=ax, width=0.3, alpha=0.3)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=15)
    nx.draw_networkx_labels(G, pos, ax=ax, font_size=5)

    if rutaDFS:
        edges_dfs = [(rutaDFS[i], rutaDFS[i+1]) for i in range(len(rutaDFS)-1)]
        nx.draw_networkx_edges(G, pos, ax=ax, edgelist=edges_dfs, width=2)

    plt.tight_layout()
    plt.show()

```


Caso 1: Observatorio - Ciudad Azteca

Ejecutamos:

```
pruebas("Observatorio","Ciudad Azteca")
```

Salidas:

En consola.

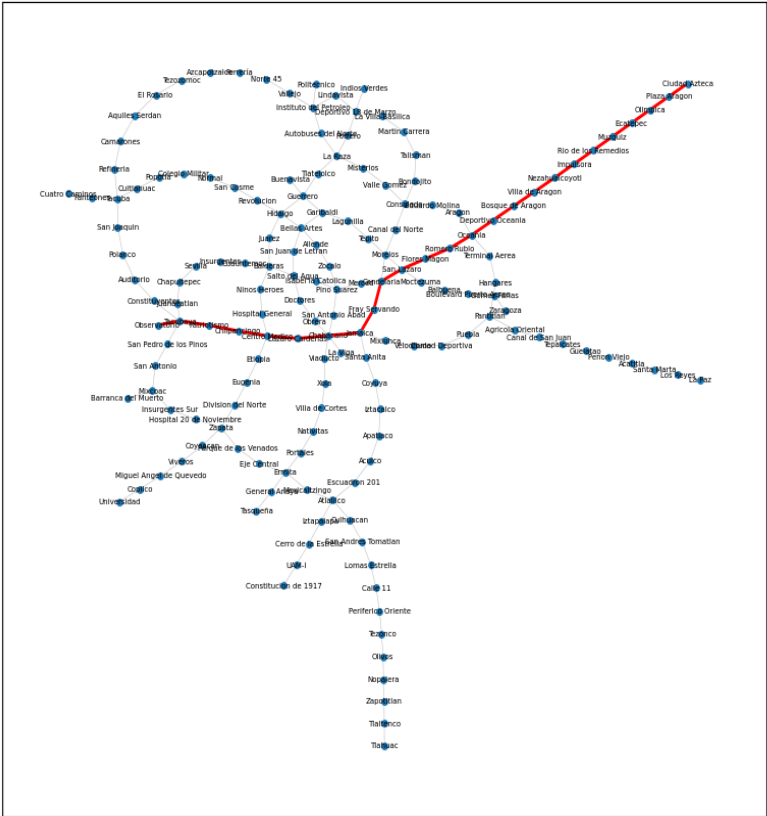
```
Resultados BFS ruta desde Observatorio hasta Ciudad Azteca:  
Observatorio -> Tacubaya -> Patriotismo -> Chilpancingo -> Centro Medico -> Lazaro Cardenas -> Chabacano -> Jamaica -> Fray Servando  
-> Candelaria -> San Lazaro -> Flores Magon -> Romero Rubio -> Oceania -> Deportivo Oceania -> Bosque de Aragon -> Villa de Aragon  
-> Nezahualcoyotl -> Impulsora -> Rio de los Remedios -> Muzquiz -> Ecatepec -> Olimpica -> Plaza Aragon -> Ciudad Azteca  
Estaciones de distancia: 24  
Cantidad de nodos expandidos: 162  
Resultados DFS ruta desde Observatorio hasta Ciudad Azteca:  
Observatorio -> Tacubaya -> Patriotismo -> Chilpancingo -> Centro Medico -> Lazaro Cardenas -> Chabacano -> Jamaica -> Mixiuhca ->  
Velodromo -> Ciudad Deportiva -> Puebla -> Pantitlan -> Hangares -> Terminal Aerea -> Oceania -> Deportivo Oceania -> Bosque de  
Aragon -> Villa de Aragon -> Nezahualcoyotl -> Impulsora -> Rio de los Remedios -> Muzquiz -> Ecatepec -> Olimpica -> Plaza Aragon  
-> Ciudad Azteca  
Estaciones de distancia: 26  
Cantidad de nodos expandidos: 35
```

Comparativa de rendimiento:

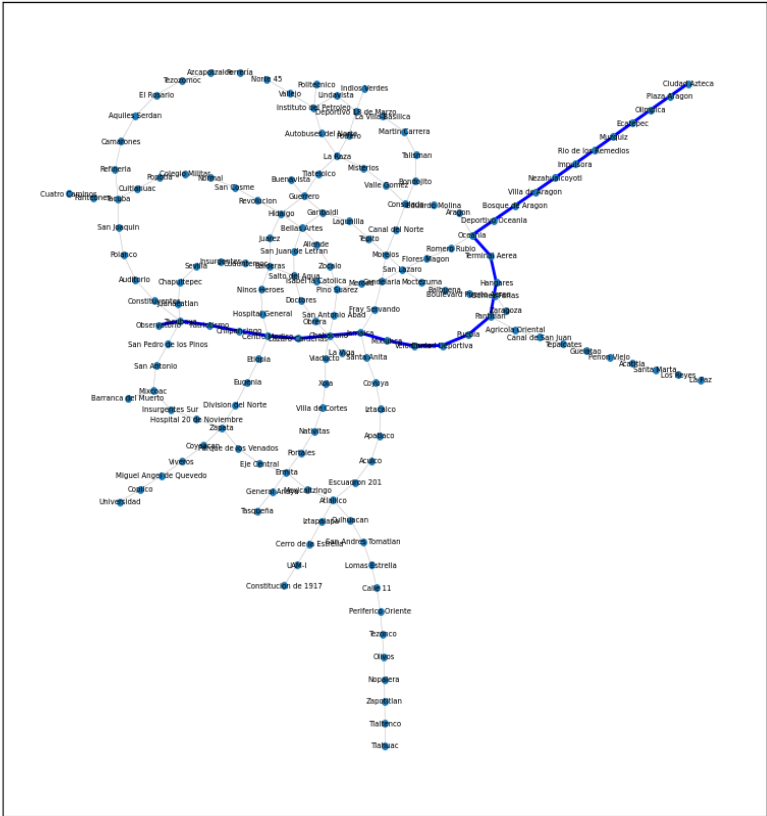
	BFS	DFS
Distancia	24	26
Nodos Expandidos	162	35

Gráficamente:

BFS: Observatorio -> Ciudad Azteca



DFS: Observatorio -> Ciudad Azteca



Caso 2: Indios Verdes - Velódromo

Ejecutamos:

```
#Prueba 2

pruebas("Indios Verdes","Velodromo")
```

Salidas:

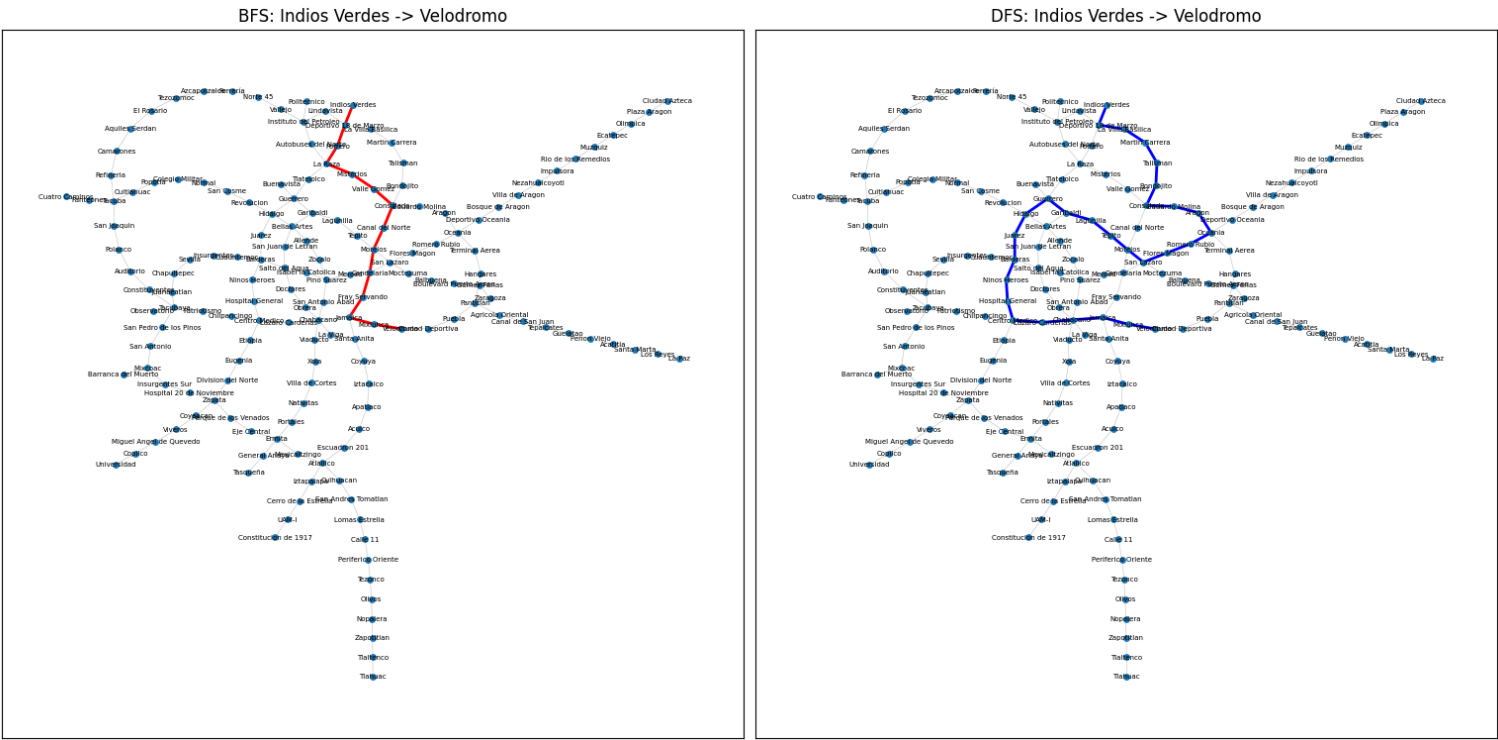
En consola:

```
Resultados BFS ruta desde Indios Verdes hasta Velodromo:
Indios Verdes -> Deportivo 18 de Marzo -> Potrero -> La Raza -> Misterios -> Valle Gomez -> Consulado -> Canal del Norte -> Morelos
-> Candelaria -> Fray Servando -> Jamaica -> Mixiuhca -> Velodromo
Estaciones de distancia: 13
Cantidad de nodos expandidos: 90
Resultados DFS ruta desde Indios Verdes hasta Velodromo:
Indios Verdes -> Deportivo 18 de Marzo -> La Villa-Basilica -> Martin Carrera -> Talisman -> Bondojoito -> Consulado -> Eduardo
Molina -> Aragon -> Oceania -> Romero Rubio -> Flores Magon -> San Lazaro -> Morelos -> Tepito -> Lagunilla -> Garibaldi -> Guerrero
-> Hidalgo -> Juarez -> Balderas -> Ninos Heroes -> Hospital General -> Centro Medico -> Lazaro Cardenas -> Chabacano -> Jamaica ->
Mixiuhca -> Velodromo
Estaciones de distancia: 28
Cantidad de nodos expandidos: 40
```

Comparativa de rendimiento:

	BFS	DFS
Distancia	13	28
Nodos Expandidos	90	40

Gráficamente:



Caso 3: UAM-I - Rosario

Ejecutamos:

```
#Prueba 3
pruebas("UAM-I","El Rosario")
```

Salidas:

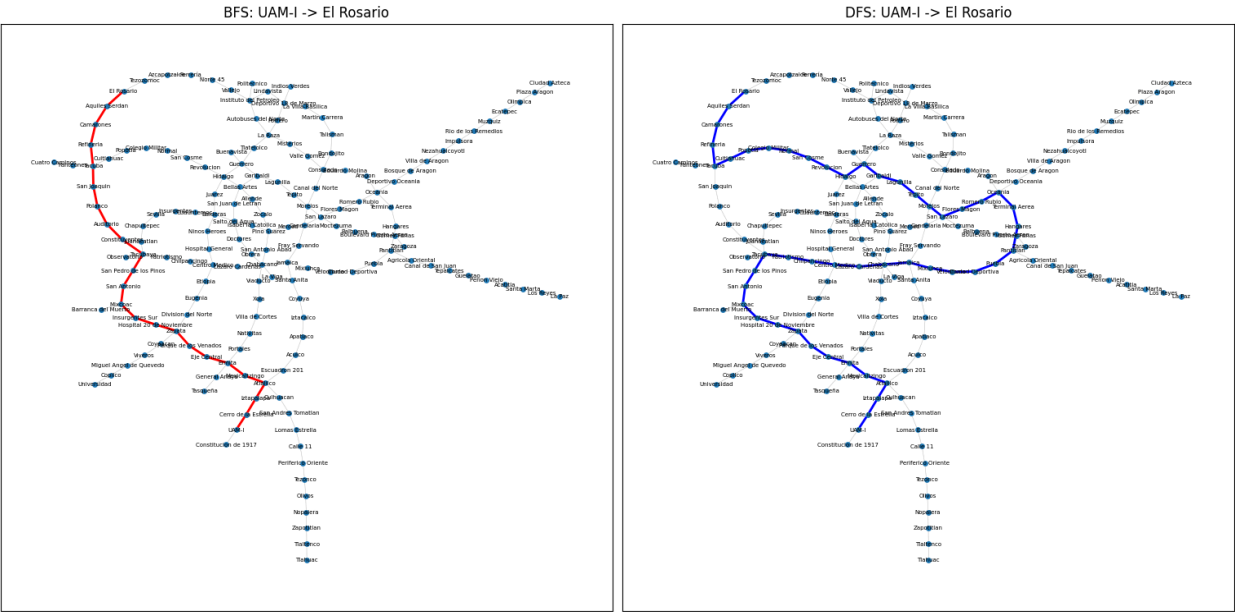
En consola:

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 1\main.py"
Resultados BFS ruta desde UAM-I hasta El Rosario:
UAM-I -> Cerro de la Estrella -> Iztapalapa -> Atlalilco -> Mexicaltzingo -> Ermita -> Eje Central -> Parque de los Venados ->
Zapata -> Hospital 20 de Noviembre -> Insurgentes Sur -> Mixcoac -> San Antonio -> San Pedro de los Pinos -> Tacubaya ->
Constituyentes -> Auditorio -> Polanco -> San Joaquin -> Tacuba -> Refineria -> Camarones -> Aquiles Serdan -> El Rosario
Estaciones de distancia: 23
Cantidad de nodos expandidos: 155
Resultados DFS ruta desde UAM-I hasta El Rosario:
UAM-I -> Cerro de la Estrella -> Iztapalapa -> Atlalilco -> Mexicaltzingo -> Ermita -> Eje Central -> Parque de los Venados ->
Zapata -> Hospital 20 de Noviembre -> Insurgentes Sur -> Mixcoac -> San Antonio -> San Pedro de los Pinos -> Tacubaya -> Patriotismo
-> Chilpancingo -> Centro Medico -> Lazaro Cardenas -> Chabacano -> Jamaica -> Mixiuhca -> Velodromo -> Ciudad Deportiva -> Puebla
-> Pantitlan -> Hangares -> Terminal Aerea -> Oceania -> Romero Rubio -> Flores Magon -> San Lazaro -> Morelos -> Tepito ->
Lagunilla -> Garibaldi -> Guerrero -> Hidalgo -> Revolucion -> San Cosme -> Normal -> Colegio Militar -> Popotla -> Cuitlahuac ->
Tacuba -> Refineria -> Camarones -> Aquiles Serdan -> El Rosario
Estaciones de distancia: 48
Cantidad de nodos expandidos: 108
```

Comparativa de rendimiento:

	BFS	DFS
Distancia	23	155
Nodos Expandidos	48	108

Gráficamente:



4. Analisis Teorico

4.1 Análisis de Completitud

"Un algoritmo es completo si garantiza encontrar una solución siempre que exista una."

Russell, Stuart; Norvig, Peter. Artificial Intelligence: A Modern Approach, 3rd ed., Pearson, 2010.

Siguiendo esta definición entendemos que nuestros DFS y BFS serian completos si cada que proporcionamos un problema con al menos una solución existente este nos regresa una, aunque podemos presumir que para cada prueba manual esto es cierto se creó la siguiente función para comprobar la completitud bajo distintos casos válidos que definimos aquí como: "Cualquier par de estaciones existentes como inicio y final respectivamente"

```
def pruebasSinGraficar(inicio, fin):  
  
    # BFS  
    rutaBFS, visitados_bfs = bfs.bfs(grafoMetro.metro, inicio, fin)  
  
    # DFS  
    rutaDFS, visitados_dfs = dfs.dfs(grafoMetro.metro, inicio, fin)  
  
    if rutaBFS is not None and rutaDFS is not None:  
        return True  
  
    return False
```

Esta función simplemente regresa false si para cualquier caso de dos estaciones dentro del grafo "metro" no encuentra una solución, o sea un camino de una estación a otra, posteriormente en nuestro "main.py" se ejecuta la siguiente secuencia haciendo uso del módulo "random".

```
numPruebas = 100000  
n = 0  
  
while n < numPruebas:  
    n = n+1  
    inicioAleatorio, finalAleatorio = estacionesAleatorias()  
    completitud = pruebasSinGraficar(inicioAleatorio, finalAleatorio)  
    if completitud is not True:  
        break  
  
if completitud:  
    print(f"Cumple para #{n} pruebas")  
else:  
    print(f"No cumple para #{n} pruebas")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Z/OS CONSOLE
[Done] exited with code=1 in 9.806 seconds

[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 1\main.py"
Cumple para ☐ #100000 pruebas

[Done] exited with code=0 in 8.99 seconds
```

Podemos decir entonces que para cada prueba en la que se proporcionan estaciones (inicios y destinos válidos) ambos algoritmos cumplen con completitud.

En caso de grafos infinitos

Ambos casos pierden su completitud ante distintos tipos de grafo infinito, en el caso de DFS imaginemos un grafo cuya solución es el primer nodo a la derecha del nodo raíz pero su algoritmo invita a empezar la exploración del grafo por la izquierda que se expande infinitamente, para ese grafo DFS jamás encontrará una solución.

Para BFS en cambio, el problema es la ramificación infinita, imaginemos un grafo cuya solución se encuentra en el tercer nivel pero su segundo nivel se expande infinitamente, en ese caso jamás alcanzaría a explorar el nivel de la solución y perdería su completitud, aunque ambos son ejemplos básicos, representan fundamentalmente los problemas que ambos tienen al encontrarse con grafos infinitos.

Utilidad de visitados

Para ambos casos visitados juega un papel fundamental puesto que define cuáles nodos no hay que visitar de nuevo y por lo tanto nos evita el caer en ciclos y avanzar hacia nuevos estados, a su vez se cerciora que el algoritmo termine en el caso de haber explorado el grafo por completo asegurando su completitud si es que existe una solución.

4.2 Optimalidad

"Un algoritmo de búsqueda es óptimo si siempre encuentra una solución de costo mínimo entre todas las soluciones posibles."

Russell, Stuart; Norvig, Peter. Artificial Intelligence: A Modern Approach, 3rd ed., Pearson, 2010.

En el contexto de nuestros algoritmos, BFS es óptimo siempre que todas las aristas del grafo tengan el mismo costo, ya que explora los nodos por niveles y encuentra primero el camino con el menor número de transiciones entre estaciones. Por esta razón, en el grafo del metro, donde cada conexión puede considerarse de costo uniforme, BFS garantiza el camino más corto en número de estaciones.

Por otro lado, DFS no es un algoritmo óptimo, ya que explora en profundidad una rama sin considerar si existen caminos más cortos en otras partes del grafo. Como resultado, puede encontrar una solución válida pero no necesariamente la de menor costo.

4.3 Complejidad en Tiempo

"El tiempo de ejecución de un algoritmo se mide en función del tamaño de la entrada y corresponde al número de operaciones elementales que realiza."

Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford. Introducción a los algoritmos, 3ra ed., MIT Press / McGraw-Hill Interamericana, 2009.

La complejidad en tiempo de los algoritmos de búsqueda en grafos se determina en función del número de nodos $|V|$ y del número de aristas $|E|$ que componen el grafo, tanto BFS como DFS exploran el grafo visitando cada nodo a lo mucho una vez y revisando las conexiones de cada uno. Durante la ejecución, cada nodo se marca como visitado y cada arista es examinada cuando se recorren los vecinos del nodo, sin embargo, el resultado final de la complejidad depende de cómo esté representado el grafo y es otra de las razones para utilizar lista de adyacencia en lugar de matriz:

Lista de Adyacencia: En el peor caso, ambos algoritmos deben recorrer todos los nodos y todas las aristas. Esto da como resultado una complejidad lineal respecto al tamaño del grafo: $O(|V| + |E|)$.

Matriz de Adyacencia: Si el grafo se representa en una matriz, para cada nodo es necesario revisar la fila completa de conexiones (tengan aristas o no). Esto eleva la complejidad a una forma cuadrática: $O(|V|^2)$

4.4 Complejidad en Espacio

"Para BFS, el espacio es un problema mayor que el tiempo... Para DFS, los requerimientos de memoria son mucho más modestos."

— Russell, Stuart; Norvig, Peter. Artificial Intelligence: A Modern Approach, 3rd ed., Pearson, 2010.

Mientras que la complejidad en tiempo mide el esfuerzo de procesamiento, la complejidad en espacio mide la cantidad máxima de memoria necesaria para almacenar los nodos durante la búsqueda, tanto en "frontera" como en el set de "visitados".

Para BFS:

Debido a que BFS explora nivel por nivel, debe mantener en memoria todos los nodos del nivel actual y los nodos del siguiente nivel en la "frontera". En grafos con un factor de ramificación alto, la cantidad de nodos crece exponencialmente.

$$O(b^d)$$

Donde “b” es el factor de ramificación y “d” la profundidad a la que se encuentra la solución.

Para DFS:

A diferencia de BFS, DFS sólo necesita almacenar una única ruta desde la raíz hasta el nodo hoja, junto con los nodos hermanos que aún no han sido expandidos en cada nivel de esa ruta.

$$O(b * m)$$

Donde “b” es el factor de ramificación y “m” la profundidad máxima del árbol.

5. Conclusión

Aunque realmente esto fue una revisita a dos algoritmos que previamente había estudiado en Estructuras de Datos y Algoritmos II, esta vez gracias al escenario sobre el que se implementó entendí la utilidad práctica que tienen estos algoritmos en la vida real pese a que son dos de los algoritmos de búsqueda en grafos más básicos, gracias a los ejemplos vistos en clase me fue muy intuitivo entender cómo funcionaba el barrido de niveles de BFS contra la ida a profundidad de DFS, las limitaciones de ambos son muy evidentes también, BFS tiene un gran problema cuando las ramificaciones de nivel a nivel aumentan en gran medida, y pese a que asegura siempre encontrar un camino (a excepción del mencionado caso de un grafo con ramificación infinita) entiendo que la complejidad en memoria es lo que limita más a este algoritmo, por otra parte DFS me pareció mucho menos elegante por la incapacidad de dar rutas óptimas en la mayoría de las ocasiones, curiosamente una de las primeras rutas que utilicé para depurar el código fue **Observatorio - Pantitlan** para la cual DFS (supongo que por cómo definí mi grafo) encuentra la ruta óptima en el menor número de saltos posibles:

```
Resultados DFS desde Observatorio hasta Pantitlan:  
Observatorio -> Tacubaya -> Patriotismo -> Chilpancingo  
Puebla -> Pantitlan  
Saltos: 12  
Nodos expandidos: 12
```

Lo que demuestra que, si se alinea todo, este algoritmo puede derrotar en ciertas ocasiones a BFS.

Por último me resulta curioso que nunca había considerado a estos algoritmos como “Inteligencia Artificial” por la idea de IA que tenemos todos hoy en día pero viéndolo en retrospectiva son agentes de búsqueda capaces de encontrar soluciones por sí solos.

Referencias.

- Russell, Stuart; Norvig, Peter. Artificial Intelligence: A Modern Approach, 3rd ed., Pearson, 2010.
- <https://cienciadedatos.net/documentos/pygml01-introduccion-grafos-redes-python>
- Romero. A. Inteligencia Artificial. Clase 3: Algoritmos de Búsqueda (AIMA)